Machine Learning and Big Data Processing

Projects 2026

Nikolaos Deligiannis, Adrian Munteanu, Evangelia Tsiligianni (course titulars)

William Martis, Leandro Di Bella (TAs)

Guidelines

The aim of the project is to study, implement, test and demonstrate deep learning algorithms. For your implementations, you should use Python with deep learning libraries (e.g., PyTorch, HuggingFace Transformers).

- You have to write your own Python code. Additional libraries/open-source source codes can be used but may not constitute the bulk of your project.
- Directly copying existing code from external sources without acknowledgement disqualifies the project.
- Each group consists of 3 members.
- You have to submit a written project report of 4–6 pages.
- Sections (obligatory): Introduction, State of the Art, Description of your algorithm/approach, Experimental results, Conclusions, References.

Overview

The Course Project is an opportunity for you to apply what you have learned in class to a problem of your interest. We have compiled a list of projects that combine recent trends and interesting applications. Please feel free to choose one of the project topics listed below.

To inspire ideas on network architecture or dataset, you might also look at recent deep learning publications from top-tier conferences: CVPR, ICCV, ECCV, NeurIPS, ICLR, ICML, as well as Kaggle challenges and the datasets listed at the end of this document.

Honor Code

You may consult any papers, books, online references, or publicly available implementations for ideas and code that you may want to incorporate into your strategy or algorithm, so long as you clearly cite your sources in your code and your writeup. However, under no circumstances may you look at another group's code or incorporate their code into your project.

Final Report

Your final write-up is required to be between 4-6 pages using the provided template, structured like a paper from an IEEE journal (Double column paper). The template can be found here: https://www.ieee.org/conferences/publishing/templates

Suggested structure:

- Title, Author(s)
- Abstract (max 300 words): Briefly describe your problem, approach, and key results.
- Introduction (10%): Describe the problem, why it's important, and an overview of your results.
- Related Work (10%): Discuss published work that relates to your project.
- Data (10%): Describe the data you are working with.
- Methods (30%): Discuss your approach and justify your design choices.
- Experiments (30%): Demonstrate that your approach solves the problem with graphs, tables, ablations.
- Conclusion (5%): Summarize key results and suggest future directions.
- Writing / Formatting (5%): Clear writing and proper formatting.

- **Project submission**
  Submit a **pdf** report, along with the code and a README.txt file describing how to run the code (all in a zip file) to: leandro.di.bella@vub.be , william.arthur.martis@vub.be

- **Grading**

The project accounts for **30%** of the final score for the exam.
The project grading is based on:

- Technical quality (score per group)- 35%
- Written report (score per group)- 25%
- Oral defense (individual score)- 40%

The oral presentation and Q&A (30 minutes of which 15-20min presentation) will be done in groups with a single Power point presentation. Your score will be based on your understanding of the machine learning and big data processing techniques you used in your project.
The selection of a topic and group will be available starting from Friday **April 3rd (20:00)** (exact procedure is described in the introduction slides) and will be closed **Sunday, April 5th (23:59)**.
The project submission deadline is on Monday, **1st of April 2025**. Please note that a late submission within 24 hours after the deadline (thus before Tuesday, 2snd of June (23:59)) will automatically lead to a -2 on the final project score. Any submission after this 'grace period' will NOT be accepted.
The date of the oral defense will be announced once determined.

Computer Vision

1. Image Super-Resolution

Difficulty: Medium

Description

Super-resolution from a single low-resolution image is a classic ill-posed problem in computer vision. Modern approaches train deep neural networks on paired low-resolution / high-resolution image datasets. The network learns to hallucinate plausible high-frequency details that are missing from the low-resolution input. A key design choice is the upsampling strategy: sub-pixel convolution (which interleaves pixels from multiple feature maps), transposed convolution, or working on a pre-upsampled bicubic input. Residual connections have proven essential for stable training at high scale factors.

Objectives

The first step is to retrieve the DIV2K dataset, which provides high-quality 2K images and their pre-computed low-resolution counterparts at multiple scale factors (×2, ×4). You should implement at least two upsampling strategies (e.g., sub-pixel convolution vs. transposed convolution) and compare them. After training, compute PSNR and SSIM on the validation set. Upon completing this project, we expect: (i) a brief description of the literature; (ii) a working super-resolution network with at least two different architecture variants; (iii) an experimental study comparing reconstruction quality (PSNR, SSIM) against bicubic interpolation and across scale factors.

References

- Lim, B., Son, S., Kim, H., Nah, S., Lee, K.M. "Enhanced Deep Residual Networks for Single Image Super-Resolution." CVPR Workshops, pp. 136–144, 2017.
- Shi, W., Caballero, J., Huszár, F., et al. "Real-Time Single Image and Video Super-Resolution Using an Efficient Sub-Pixel Convolutional Neural Network." CVPR, pp. 1874–1883, 2016.
- Dong, C., Loy, C.C., He, K., Tang, X. "Learning a Deep Convolutional Network for Image Super-Resolution." ECCV, pp. 184–199, 2014.
- Dataset: https://data.vision.ee.ethz.ch/cvl/DIV2K/

---

2. Image Inpainting

Difficulty: Medium

Description

Image inpainting aims to fill in missing or corrupted regions of an image so that the result looks natural and coherent with the surrounding content. Missing regions can take many forms: a square crop in the center, randomly scattered blocks, or irregular free-form masks. Traditional approaches relied on patch matching and diffusion-based texture synthesis. Modern methods use convolutional autoencoders, often trained adversarially with a GAN discriminator that forces the generator to produce realistic completions rather than blurry averages.

Objectives

You should use ImageNet (subset) or Places365 as your image source, and generate masks yourself (center crop, random blocks, or free-form masks from the Liu et al. irregular mask dataset). Implement a convolutional autoencoder as a baseline, then extend it with adversarial training (GAN). Compare results qualitatively and quantitatively (PSNR, SSIM, FID if feasible). Upon completing this project, we expect: (i) a brief description of the literature; (ii) a working inpainting model with at least an autoencoder and a GAN variant; (iii) experimental evaluations showing how mask type and size affect reconstruction quality.

References

- Pathak, D., Krahenbuhl, P., Donahue, J., Darrell, T., Efros, A.A. "Context Encoders: Feature Learning by Inpainting." CVPR, pp. 2536–2544, 2016.
- Yu, J., Lin, Z., Yang, J., Shen, X., Lu, X., Huang, T.S. "Generative Image Inpainting with Contextual Attention." CVPR, 2018.
- Yang, C., Lu, X., Lin, Z., Shechtman, E., Wang, O., Li, H. "High-Resolution Image Inpainting Using Multi-Scale Neural Patch Synthesis." CVPR, pp. 4076–4084, 2017.

---

3. Grayscale Image Colorization

Difficulty: Medium

Description

A large amount of historical grayscale imagery exists online. Automatically adding plausible color is a challenging task because the problem is inherently multimodal: a grayscale pixel could correspond to many different colors. Modern approaches formulate colorization as a per-pixel classification problem in the CIE Lab color space. The network takes the L (luminance) channel as input and predicts a distribution over quantized ab (chrominance) values. Because most natural images are dominated by low-saturation colors, a class-rebalancing scheme is essential to encourage the model to predict vibrant colors.

Objectives

Build a grayscale/color image dataset from a public source such as ImageNet or the Berkeley Segmentation Dataset. Convert images to Lab space. Implement a CNN that predicts the ab channels from the L channel. Experiment with both regression and classification formulations, and study the effect of class rebalancing on color vividness. Upon completing this project, we expect: (i) a brief description of the literature; (ii) a working colorization model with at least two formulation variants (regression vs. classification); (iii) qualitative and quantitative comparisons showing the impact of class rebalancing and architecture choices.

References

- Zhang, R., Isola, P., Efros, A.A. "Colorful Image Colorization." ECCV, pp. 649–666, 2016.
- Cheng, Z., Yang, Q., Sheng, B. "Deep Colorization." ICCV, pp. 415–423, 2015.
- Nazeri, K., Ng, E., Ebrahimi, M. "Image Colorization using Generative Adversarial Networks." arXiv:1803.05400, 2018.

---

4. Damaged Building Assessment from Satellite Imagery

Difficulty: Medium-Hard

Description

After a natural disaster, rapid assessment of building damage is critical for emergency response. Traditionally, this requires in-person inspections within 24–48 hours, which is dangerous and slow. Satellite imagery combined with deep learning offers an alternative: a model can classify each building's damage level by comparing pre- and post-disaster images. The challenge lies in handling multiple disaster types (earthquakes, floods, hurricanes), severe class imbalance (most buildings are undamaged), and variable image quality.

Objectives

Use the xBD dataset, which provides pre- and post-event satellite imagery with building polygons and ordinal damage labels (no damage / minor / major / destroyed). You may work on a subset of disaster events. Implement a change-detection architecture (e.g., a Siamese network or a U-Net with dual inputs) that takes both pre- and post-disaster images. Evaluate using the official weighted F1 metric from the xView2 challenge. Upon completing this project, we expect: (i) a brief description of the literature on satellite-based damage assessment; (ii) a working model that takes pre/post image pairs and outputs damage classifications; (iii) experimental evaluations across disaster types, with analysis of class imbalance handling strategies.

References

- Gupta, R., Goodman, B., Patel, N., et al. "Creating xBD: A Dataset for Assessing Building Damage from Satellite Imagery." CVPR Workshops, pp. 10–17, 2019.
- Weber, E., Kané, H. "Building Disaster Damage Assessment in Satellite Imagery with Multi-Temporal Fusion." arXiv:2004.05525, 2020.
- Dataset: https://xview2.org/

---

5. Crop Disease Diagnosis from Leaf Photos

Difficulty: Easy

Description

Crop diseases threaten food security globally. Early detection is essential for timely treatment. Smartphone-based diagnosis using deep learning could make expert-level detection accessible to farmers everywhere. The core challenge is a classification task: given a photograph of a plant leaf, determine which disease (if any) is present. However, a key issue is domain shift; models trained on clean, lab-collected images may fail on noisy, poorly lit field photos.

Objectives

Use the PlantVillage dataset (54,306 images, 14 crop species, 38 classes). Train a CNN or fine-tune a pretrained model (ResNet, EfficientNet) for disease classification. Investigate the effect of training on the original colored images vs. grayscale vs. segmented versions. Evaluate generalization by testing on images from a different source or with synthetic noise/blur to simulate real-world conditions. Upon completing this project, we expect: (i) a brief description of the literature; (ii) a working classification model with multiple architecture comparisons; (iii) an analysis of robustness to domain shift and the effect of data augmentation.

References

- Mohanty, S.P., Hughes, D.P., Salathé, M. "Using Deep Learning for Image-Based Plant Disease Detection." Frontiers in Plant Science 7, 2016.
- Dataset: https://github.com/spMohanty/PlantVillage-Dataset

---

6. Deepfake Face Detection

Difficulty: Medium

Description

The rapid progress in face synthesis and manipulation (deepfakes, face swaps, facial reenactment) poses a growing threat to media integrity. A deepfake detector must distinguish authentic face images or video frames from manipulated ones. The difficulty is compounded by video compression, which destroys the subtle artifacts that detectors rely on. A robust system should generalize across manipulation methods and remain effective at different compression levels.

Objectives

Use the FaceForensics dataset, which provides 1,000 original YouTube videos manipulated with four methods (DeepFakes, Face2Face, FaceSwap, NeuralTextures) at three compression levels (raw, light c23, heavy c40). Train a binary forgery classifier (real vs. fake) and evaluate per-method and per-compression performance. Experiment with at least two backbone architectures (e.g., XceptionNet and EfficientNet). Evaluate cross-manipulation generalization. Upon completing this project, we expect: (i) a brief description of the deepfake detection literature; (ii) a working detector evaluated at multiple compression levels; (iii) an analysis of cross-method generalization and failure cases.

References

- Rössler, A., Cozzolino, D., Verdoliva, L., et al. "FaceForensics++: Learning to Detect Manipulated Facial Images." ICCV, 2019.
- Yan, Z., Zhang, Y., Yuan, X., et al. "DeepfakeBench: A Comprehensive Benchmark of Deepfake Detection." NeurIPS Datasets and Benchmarks, 2023.
- Dataset: https://huggingface.co/datasets/RohanRamesh/faceforensics-images

7. Content-Based Image Retrieval

Difficulty: Medium

Description

Content-based image retrieval (CBIR) aims to find images in a database that are visually similar to a query image. Traditional methods relied on handcrafted features (SIFT, VLAD, Bag of Words), but CNN and ViT features have dramatically improved retrieval accuracy. The system has two components: a feature extractor that maps images to compact vectors, and a retrieval component that ranks database images by distance to the query.

Objectives

Use a standard CBIR benchmark such as Oxford5k Buildings, Paris6k Buildings, or the INRIA Holidays dataset. Implement at least two feature extraction methods: one classical (e.g., VLAD with SIFT) and one deep (e.g., features from a pretrained ResNet or ViT). Compare retrieval performance using mean Average Precision (mAP). Experiment with different pooling strategies (MAC, GeM) and distance metrics. Upon completing this project, we expect: (i) a brief description of the CBIR literature; (ii) a working retrieval system with multiple feature extraction options; (iii) experimental evaluations comparing classical vs. deep features and different distance measures.

References

• Razavian, A.S., Azizpour, H., Sullivan, J., Carlsson, S. "CNN Features Off-the-Shelf: An Astounding Baseline for Recognition." CVPR Workshops, pp. 512–519, 2014.
• Radenoć, F., Tolias, G., Chum, O. "Fine-tuning CNN Image Retrieval with No Human Annotation." TPAMI, 2019.

---

8. Visual Anomaly Detection in Manufacturing

Difficulty: Medium

Description

Quality control in manufacturing requires detecting defective products, but defective samples are rare and unpredictable, a factory may encounter defect patterns it has never seen before. Unsupervised anomaly detection addresses this by training exclusively on defect-free images. At test time, the model must detect and localize anything that deviates from normality. Recent feature-matching approaches like PatchCore build a memory bank of patch features extracted from a pretrained backbone, and flag test patches whose distance to the nearest normal neighbor exceeds a threshold.

Objectives

Use the MVTec AD dataset (5,354 images, 15 categories, pixel-precise defect masks). Training sets contain only defect-free images; test sets include over 70 defect types (scratches, dents, contaminations). Implement PatchCore and compare it with at least one alternative (e.g., autoencoder). Evaluate at image level (AUROC) and pixel level (AUROC, PRO metric). Analyze per-category performance and the effect of backbone choice and memory bank size on accuracy vs. inference speed. Upon completing this project, we expect: (i) a brief description of the literature; (ii) a working anomaly detection and localization system with at least two approaches; (iii) per-category experimental evaluations with analysis of failure cases.

References

• Roth, K., Pemula, L., Zepeda, J., Schölkopf, B., Brox, T., Gehler, P. "Towards Total Recall in Industrial Anomaly Detection." CVPR, pp. 14318–14328, 2022.
• Bergmann, P., Fauser, M., Sattlegger, D., Steger, C. "MVTec AD; A Comprehensive Real-World Dataset for Unsupervised Anomaly Detection." CVPR, 2019.
• Dataset: https://www.mvtec.com/company/research/datasets/mvtec-ad

---

10. Video Action Recognition

Difficulty: Medium-Hard

Description

Video understanding requires reasoning about both spatial content (what objects are present) and temporal dynamics (how they move and interact over time). Action recognition; classifying a short video clip into one of many activity categories; is the foundational task. Early deep learning approaches applied 2D CNNs frame-by-frame and aggregated predictions. 3D CNNs (C3D, I3D) process spatiotemporal volumes directly. More recently, Video Transformers (TimeSformer, VideoMAE) treat video as a sequence of patch tokens across space and time.

Objectives

Use the UCF-101 dataset (13,320 videos, 101 action categories, 3 train/test splits) or a subset of Kinetics-400. Implement at least two approaches: a frame-level CNN baseline (e.g., ResNet features averaged over frames) and a temporal model (3D CNN). Compare their top-1 accuracy and per-class performance. Analyze which actions benefit most from temporal modeling. Upon completing this project, we expect: (i) a brief description of the video understanding literature; (ii) a working action recognition system with spatial-only and spatiotemporal variants; (iii) experimental evaluations comparing approaches, with analysis of which actions require temporal context.

References

- Tran, D., Bourdev, L., Fergus, R., Torresani, L., Paluri, M. "Learning Spatiotemporal Features with 3D Convolutional Networks." ICCV, pp. 4489–4497, 2015. (C3D)
- Tong, Z., Song, Y., Wang, J., Wang, L. "VideoMAE: Masked Autoencoders are Data-Efficient Learners for Self-Supervised Video Pre-Training." NeurIPS, 2022.
- Dataset: https://www.crcv.ucf.edu/data/UCF101.php

---

11. Face Generation with GANs

Difficulty: Medium-Hard

Description

Generative Adversarial Networks (GANs) are one of the most influential ideas in deep generative modeling. A generator network learns to produce realistic images from random noise, while a discriminator network learns to distinguish real images from generated ones. The two networks are trained in a minimax game. Despite their elegance, GANs are notoriously hard to train; mode collapse, training instability, and vanishing gradients are common failure modes. Understanding these dynamics firsthand is the core learning objective of this project.

Objectives

Use the CelebA dataset (200k celebrity face images, aligned and cropped to 64×64 or 128×128). Implement a DCGAN from scratch as your baseline. Then implement at least one improved variant: WGAN-GP or Progressive GAN. You must write the training loop yourself, do not use a pre-built GAN library. Compare the variants in terms of training stability (loss curves, mode collapse frequency), visual quality, and FID score. Upon completing this project, we expect: (i) a brief description of the GAN literature and training dynamics; (ii) a working GAN trained from scratch with at least two variants; (iii) an experimental study of training stability and image quality, with generated samples and FID scores.

References

- Goodfellow, I., Pouget-Abadie, J., Mirza, M., et al. "Generative Adversarial Nets." NeurIPS, pp. 2672–2680, 2014.
- Radford, A., Metz, L., Chintala, S. "Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks." ICLR, 2016. (DCGAN)
- Gulrajani, I., Ahmed, F., Arjovsky, M., Dumoulin, V., Courville, A. "Improved Training of Wasserstein GANs." NeurIPS, 2017. (WGAN-GP)
- Dataset: https://mmlab.ie.cuhk.edu.hk/projects/CelebA.html or https://huggingface.co/datasets/abhiyanta/celeba-dataset

Audio & Music

12. Voice Isolation from Music

Difficulty: Medium-Hard

Description

In human perception, isolating a sound source; even in a crowded environment; is natural. Replicating this for music (separating the singer's voice from instruments) is a challenging signal processing task. Modern approaches work on a spectrogram representation of the audio: a neural network learns to produce a mask for each time-frequency bin, indicating whether it belongs to the vocals or the accompaniment.

Objectives

Use the MUSDB18 dataset, which provides 150 full-length music tracks with isolated stems (vocals, drums, bass, other). Compute spectrograms using Short-Time Fourier Transform (librosa or torchaudio). Implement a U-Net-based masking model on spectrograms. Optionally compare with a waveform-domain model. Evaluate using Signal-to-Distortion Ratio (SDR). Upon completing this project, we expect: (i) a brief description of the source separation literature; (ii) a working voice isolation demo; (iii) experimental evaluations with different model architectures.

References

- Stoller, D., Ewert, S., Dixon, S. "Wave-U-Net: A Multi-Scale Neural Network for End-to-End Audio Source Separation." ISMIR, 2018.
- Jansson, A., Humphrey, E., Montecchio, N., et al. "Singing Voice Separation with Deep U-Net Convolutional Networks." ISMIR, pp. 745–751, 2017.
- Dataset: https://sigsep.github.io/datasets/musdb.html

---

13. Urban Sound Classification for Smart Cities

Difficulty: Easy

Description

Automatic classification of environmental sounds has applications in urban surveillance, noise monitoring, smart city infrastructure, and emergency detection. Unlike speech or music, environmental sounds are unstructured and highly variable. The standard approach converts audio clips to mel-spectrograms and processes them with a CNN; essentially treating audio classification as an image classification task. Data augmentation (pitch shifting, time stretching, SpecAugment) is critical given the limited size of available datasets.

Objectives

Use the UrbanSound8K dataset (8,732 labeled sound excerpts across 10 urban sound classes). You must use the predefined 10-fold cross-validation splits (do NOT reshuffle the data). Convert audio to mel-spectrograms and train a CNN classifier. Experiment with at least two data augmentation strategies. Upon completing this project, we expect: (i) a brief description of the literature; (ii) a working sound classification system with multiple augmentation strategies; (iii) experimental evaluations using 10-fold CV with confusion matrix analysis.

References

- Salamon, J., Bello, J.P. "Deep Convolutional Neural Networks and Data Augmentation for Environmental Sound Classification." IEEE Signal Processing Letters 24(3), pp. 279–283, 2017.
- Dataset: https://urbansounddataset.weebly.com/urbansound8k.html

---

14. Music Genre Classification

Difficulty: Medium

Description

Content-based music classification involves identifying characteristics of a song (genre, mood, era) from its audio content rather than metadata. A basic pipeline consists of feature extraction (mel-spectrograms, MFCCs, chroma features) followed by a classifier. Deep learning approaches typically feed mel-spectrograms directly into CNNs or CRNNs, bypassing manual feature engineering. A key challenge is that genre boundaries are subjective and fuzzy.

Objectives

Use the GTZAN Genre Collection (1,000 tracks, 10 genres, 30s each). Implement at least two approaches: one using handcrafted features (e.g., MFCCs + SVM/MLP) and one using deep learning on mel-spectrograms (CNN or CRNN). Compare their accuracy and analyze which genres are systematically confused. Upon completing this project, we expect: (i) a brief description of the literature; (ii) a working genre classification demo with at least two different approaches; (iii) an experimental study comparing feature types and classifiers.

References

- van den Oord, A., Dieleman, S., Schrauwen, B. "Deep Content-Based Music Recommendation." NeurIPS, 2013.
- Choi, K., Fazekas, G., Sandler, M., Cho, K. "Convolutional Recurrent Neural Networks for Music Classification." ICASSP, 2017.
- Dataset: http://marsyas.info/downloads/datasets.html (GTZAN)

Natural Language Processing

15. Automated Essay Grading

Difficulty: Medium

Description

Automated essay scoring (AES) uses natural language processing to evaluate the quality of student writing. The task is to predict a numeric score for an essay, ideally aligned with what a human grader would assign. Modern approaches use pretrained language models to encode essay text and then regress or classify the score. A particular challenge is that scoring rubrics vary across prompts.

Objectives

Use the ASAP-AES dataset from Kaggle (~13,000 essays across 8 prompts). Implement at least two approaches: one using handcrafted features (essay length, vocabulary richness, grammar errors) with a classical ML model, and one using a deep learning model (LSTM or fine-tuned BERT). Evaluate using Quadratic Weighted Kappa (QWK). Upon completing this project, we expect: (i) a brief description of the AES literature; (ii) a working scoring system with multiple approaches; (iii) an experimental evaluation using QWK with per-prompt analysis.

References

- Taghipour, K., Ng, H.T. "A Neural Approach to Automated Essay Scoring." EMNLP, pp. 1882–1891, 2016.
- Dataset: https://www.kaggle.com/c/asap-aes/data

---

16. Legal Clause Detection in Contracts

Difficulty: Medium-Hard

Description

Legal contract review is one of the most time-consuming tasks in corporate transactions. Lawyers must identify dozens of clause types; non-compete provisions, termination conditions, liability limitations; across contracts that can run over 100 pages. NLP models can learn to extract and classify these clauses automatically. The task is formulated as extractive question answering: given a contract and a clause type, extract the relevant text spans.

Objectives

Use the CUAD dataset (510 commercial contracts, 13,000+ annotations across 41 clause types). Fine-tune a pretrained language model (e.g., RoBERTa, DeBERTa) for extractive QA. Evaluate using AUPR and Precision@80% Recall. Analyze which clause types are easiest/hardest to detect. Upon completing this project, we expect: (i) a brief description of the legal NLP literature; (ii) a working clause extraction system; (iii) experimental evaluations per clause type with error analysis.

References

- Hendrycks, D., Burns, C., Chen, A., Ball, S. "CUAD: An Expert-Annotated NLP Dataset for Legal Contract Review." NeurIPS Datasets and Benchmarks, 2021.
- Dataset: https://huggingface.co/datasets/theatticusproject/cuad-qa

17. Fake Review Detection

Difficulty: Medium

Description

Online reviews are a primary driver of consumer purchasing decisions, but fake reviews; written by paid writers or generated by AI; undermine trust in e-commerce platforms. A fake review detector must distinguish genuine reviews from deceptive ones based on textual content, behavioral signals, or both. The challenge is that well-crafted fake reviews can be linguistically similar to genuine ones.

Objectives

Use the Ott et al. Deceptive Opinion Spam corpus (800 hotel reviews, balanced real/fake) and optionally the Yelp Fake Review dataset. Implement at least two approaches: one based on handcrafted features (TF-IDF, linguistic features) with a classical classifier, and one using a pretrained language model (BERT or RoBERTa). Analyze which linguistic signals are most predictive of deception. Upon completing this project, we expect: (i) a brief description of the fake review detection literature; (ii) a working detection system; (iii) an experimental evaluation with analysis of discriminative features.

References

- Ott, M., Choi, Y., Cardie, C., Hancock, J.T. "Finding Deceptive Opinion Spam by Any Stretch of the Imagination." ACL, pp. 309–319, 2011.
- Salminen, J., Kandpal, C., Kamel, A.M., et al. "Creating and Detecting Fake Reviews of Online Products." Journal of Retailing and Consumer Services 64, 2022.

---

18. Rumour Detection on Social Media

Difficulty: Medium-Hard

Description

Social media enables information to spread rapidly, but it also facilitates the propagation of unverified rumours. Automated rumour detection aims to classify whether a social media event is a rumour or verified information. Beyond the original post's text, the propagation structure (how quickly it spreads, reply patterns, user credibility) carries strong signals. Modern approaches model the conversation thread as a tree or graph structure.

Objectives

Use the Twitter15 and Twitter16 rumour datasets (Ma et al.) or the PHEME dataset. Implement at least two approaches: one based on text features (TF-IDF or word embeddings with an RNN/Transformer), and one incorporating propagation structure (e.g., tree-structured RNN or a graph attention network). Compare their F1 scores. Upon completing this project, we expect: (i) a brief description of the rumour detection literature; (ii) a working detection system with text-only and structure-aware variants; (iii) experimental evaluations comparing different feature types and architectures.

References

• Ma, J., Gao, W., Mitra, P., et al. "Detecting Rumors from Microblogs with Recurrent Neural Networks." IJCAI, pp. 3818–3824, 2016.
• Kochkina, E., Liakata, M., Zubiaga, A. "All-in-One: Multi-Task Learning for Rumour Verification." COLING, pp. 3402–3413, 2018.

---

19. Toxic Comment Detection

Difficulty: Medium

Description

Online platforms struggle with toxic content; comments that are threatening, insulting, obscene, or express identity-based hate. The task is multi-label classification: a single comment may simultaneously be toxic, obscene, and an insult. The challenge comes from severe class imbalance, subtle linguistic phenomena (sarcasm, coded language), and the risk of bias (models may learn to associate certain identity terms with toxicity).

Objectives

Use the Jigsaw Toxic Comment Classification dataset from Kaggle (~160k comments with multi-label annotations across 6 toxicity types). Fine-tune a pretrained language model (BERT, DistilBERT, or RoBERTa) for multi-label classification. Evaluate per-class AUC-ROC. Experiment with strategies for handling class imbalance (weighted loss, focal loss, oversampling). Upon completing this project, we expect: (i) a brief description of the toxicity detection literature; (ii) a working multi-label classifier; (iii) experimental evaluations with per-class metrics and an analysis of bias toward identity terms.

References

• Borkan, D., Dixon, L., Sorensen, J., et al. "Nuanced Metrics for Measuring Unintended Bias with Real Data for Text Classification." WWW, pp. 491–500, 2019.
• Dataset: https://www.kaggle.com/c/jigsaw-toxic-comment-classification-challenge

---

20. Sentiment Analysis with Recurrent Neural Networks

Difficulty: Easy

Description

Sentiment analysis; determining whether a piece of text expresses a positive or negative opinion; is a foundational NLP task. Recurrent architectures (RNN, GRU, LSTM) remain the best way to understand sequential modeling from first principles. An LSTM processes text token by token, maintaining a hidden state that summarizes the sequence so far. The gating mechanism controls what information to keep or discard, allowing the network to handle long-range dependencies.

Objectives

Use the Stanford Sentiment Treebank (SST-2) or the IMDB Large Movie Review dataset (50k reviews). Implement three recurrent models from scratch: a vanilla RNN, a GRU, and an LSTM. Use pretrained word embeddings (GloVe or Word2Vec). Train each model and compare their test accuracy, convergence speed, and ability to handle long reviews. Analyze failure cases involving late negation. Upon completing this project, we expect: (i) a brief description of the recurrent networks literature; (ii) from-scratch implementations of RNN, GRU, and LSTM (not

nn.LSTM; build the cell yourself); (iii) experimental comparisons on accuracy, training speed, and sensitivity to sequence length.

References

- Hochreiter, S., Schmidhuber, J. "Long Short-Term Memory." Neural Computation 9(8), pp. 1735–1780, 1997.
- Cho, K., van Merriënboer, B., Gulcehre, C., et al. "Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation." EMNLP, 2014. (GRU)
- Dataset (SST-2): https://huggingface.co/datasets/stanfordnlp/sst2
- Dataset (IMDB): https://huggingface.co/datasets/imdb

---

21. Build a Small Language Model from Scratch

Difficulty: Medium-Hard

Description

Large Language Models like GPT are built on surprisingly simple principles: a Transformer decoder trained to predict the next token in a sequence. This project strips away the scale and asks you to build a miniature GPT from scratch; small enough to train in under 30 minutes on a free Colab GPU, but large enough to generate coherent text. By implementing the architecture yourself (token embeddings, positional encodings, multi-head self-attention, feedforward layers, layer normalization), you will develop a concrete understanding of every component.

Objectives

Build a character-level or small-vocabulary Transformer decoder from scratch in PyTorch. Train it on TinyShakespeare (~1MB) or a comparable corpus. Your model should have 2–6 layers, 4–8 attention heads, and an embedding dimension of 128–256 (~1–10M parameters). Implement the training loop with teacher forcing and cross-entropy loss. Generate text samples and evaluate perplexity. Perform an ablation study: measure the effect of removing positional encoding, reducing layers, or replacing multi-head with single-head attention. Upon completing this project, we expect: (i) a brief description of the Transformer and language modeling literature; (ii) a complete from-scratch implementation (no nn.TransformerDecoder); (iii) training curves, generated text samples, perplexity measurements, and an ablation study.

References

- Vaswani, A., Shazeer, N., Parmar, N., et al. "Attention Is All You Need." NeurIPS, 2017.
- Radford, A., Wu, J., Child, R., et al. "Language Models are Unsupervised Multitask Learners." OpenAI Technical Report, 2019. (GPT-2)
- Karpathy, A. "nanoGPT"; minimal GPT training code. https://github.com/karpathy/nanoGPT
- Dataset: https://raw.githubusercontent.com/karpathy/char-rnn/master/data/tinyshakespeare/input.txt

---

Multimodal (Vision + Language)

22. Recipe Generator from Fridge Photos

Difficulty: Hard

Description

This multimodal project bridges computer vision and natural language generation. Given a photograph of fridge or pantry contents, the system must first identify the visible ingredients (a vision task), then generate plausible recipes using those ingredients (a language generation task). This requires cross-modal reasoning: the system must understand that a tomato, mozzarella, and basil suggest a caprese salad, not a chocolate cake.

Objectives

Use the Recipe1M+ dataset (1M+ recipes with 13M food images) for recipe generation, and the Food-101 or Ingredients-101 dataset for ingredient recognition. You may build the system in two stages (ingredient detector + text generator) or use a vision-language model (e.g., SmolVLM or BLIP-2) end-to-end. Finetune the VLM with Lora. Evaluate ingredient detection with precision/recall and recipe quality with BLEU/ROUGE. Upon completing this project, we expect: (i) a brief description of the multimodal recipe generation literature; (ii) a working pipeline from photo to recipe; (iii) experimental evaluations of both the vision and generation components.

References

- Salvador, A., et al. "Learning Cross-modal Embeddings for Cooking Recipes and Food Images." CVPR, 2017.
- Liu, H., Li, C., Wu, Q., Lee, Y.J. "Visual Instruction Tuning." NeurIPS, 2023. (LLaVA)
- Dataset: http://pic2recipe.csail.mit.edu/ (Recipe1M+) or https://huggingface.co/datasets/m3hrdadfi/recipe_nlg_lite

---

23. Medical Visual Question Answering; Fine-tuning a VLM with LoRA

Difficulty: Hard

Description

Vision-Language Models (VLMs) like LLaVA, Qwen2-VL, and PaliGemma can answer questions about images, but they perform poorly on medical imagery out of the box because their pretraining data lacks clinical content. Low-Rank Adaptation (LoRA) offers a parameter-efficient way to specialize these models for medical visual question answering without retraining all parameters.

Objectives

Select a pretrained VLM (SmolVLM, Qwen 0.8B). Fine-tune it with LoRA (rank 8 or 16) on VQA-RAD (~3,500 QA pairs on radiology images) and/or SLAKE (~14k QA pairs). Compare LoRA-adapted vs. zero-shot performance on open-ended and closed-ended questions. Report accuracy stratified by question type and imaging modality. Upon completing this project, we expect: (i) a brief description of the Med-VQA and parameter-efficient fine-tuning literature; (ii) a working LoRA-adapted VLM; (iii) experimental evaluations comparing zero-shot vs. fine-tuned performance with error analysis.

References

- Hu, E.J., Shen, Y., Wallis, P., et al. "LoRA: Low-Rank Adaptation of Large Language Models." ICLR, 2022.
- Lau, J.J., et al. "A Dataset of Clinically Generated Visual Questions and Answers about Radiology Images." Scientific Data 5, 2018. (VQA-RAD)
- Dataset: https://huggingface.co/datasets/flaviagiammarino/vqa-rad

---

24. Visual Fact-Checking

Difficulty: Hard

Description

Misinformation often pairs real images with false or misleading captions, creating out-of-context media that appears credible. Visual fact-checking requires determining whether an image actually supports an accompanying text claim, or if it has been repurposed from a different context. This multimodal reasoning task goes beyond simple image-text matching.

Objectives

Use the NewsCLIPpings dataset (~72k image-caption pairs, balanced between genuinely matched and out-of-context combinations). Implement a multimodal classifier that takes an image-caption pair and predicts whether they are matched or mismatched. Compare a simple CLIP-based similarity baseline with a fine-tuned multimodal model. Analyze what types of mismatches are easiest and hardest to detect. Upon completing this project, we expect: (i) a brief description of the multimodal misinformation detection literature; (ii) a working fact-checking classifier; (iii) experimental evaluations with analysis of mismatch types and failure modes.

References

- Luo, G., Darrell, T., Rohrbach, A. "NewsCLIPpings: Automatic Generation of Out-of-Context Multimodal Media." EMNLP, 2021.
- Radford, A., Kim, J.W., Hallacy, C., et al. "Learning Transferable Visual Models from Natural Language Supervision." ICML, 2021. (CLIP)
- Dataset: https://github.com/g-luo/news_clippings

Time Series & Forecasting

25. Time-Series Forecasting with Transformers

Difficulty: Medium-Hard

Description

Long-term time-series forecasting is essential for energy management, weather prediction, and infrastructure planning. PatchTST addresses this by segmenting time series into subseries-level patches (analogous to image patches in ViT) and processing them with channel-independent Transformers. This design reduces computational cost, retains local temporal semantics, and allows the model to attend to longer histories.

Objectives

Use the standard benchmarks from the Autoformer repository: ETT (Electricity Transformer Temperature), Weather (21 channels, 52k timesteps), and/or Electricity (321 channels). Implement or use a PatchTST model or similar (from sratch) and compare it against at least one baseline (ARIMA). Evaluate using MSE and MAE across multiple prediction horizons (96, 192, 336, 720 steps). Upon completing this project, we expect: (i) a brief description of the time-series forecasting literature; (ii) a working Transformer-based forecaster; (iii) experimental evaluations across datasets and horizons.

References

- Nie, Y., Nguyen, N.H., Sinthong, P., Kalagnanam, J. "A Time Series is Worth 64 Words: Long-term Forecasting with Transformers." ICLR, 2023. (PatchTST)
- Zeng, A., Chen, M., Zhang, L., Xu, Q. "Are Transformers Effective for Time Series Forecasting?" AAAI, 2023. (DLinear)
- Datasets: https://github.com/thuml/Autoformer

---

26. Trajectory Forecasting on nuScenes

Difficulty: Very Hard

Description

Predicting the future trajectories of vehicles and pedestrians is a core capability for autonomous driving. Given past motion history and rich sensor context (cameras, LiDAR, maps), the system must forecast where each agent will be over the next several seconds. The problem is inherently multimodal; an approaching car might turn left, turn right, or continue straight; so the model must produce multiple plausible future trajectories.

Objectives

Use the nuScenes dataset (mini trainval split). It provides 1,000 driving scenes with synchronized 6-camera views, LiDAR, radar, and 3D annotations at 2 Hz. Please, only focus one modality (e.g. the front camera view or lidar frame) and the historical motion trajectory of the car. Predict future trajectories over a 6-second horizon. You may explore any approach: classical motion forecasting, Transformer-based prediction (Implement a small version of a baseline) or a VLA model (VLM Qwen + output head). Evaluate using minADE, minFDE, and Miss Rate. Upon completing this project, we expect: (i) a brief description of the trajectory forecasting literature; (ii) a working prediction model; (iii) experimental evaluations with standard metrics and qualitative visualization of predicted trajectories.

References

- Caesar, H., Bankiti, V., Lang, A.H., et al. "nuScenes: A Multimodal Dataset for Autonomous Driving." CVPR, 2020.
- Chai, Y., et al. "DeMo: Decoupling Motion Forecasting into Directional Intentions and Dynamic States." NeurIPS, 2024.
- Zhou, L., et al. "OpenDriveVLA: Towards End-to-end Autonomous Driving with Large Vision Language Action Model." arXiv:2503.23463, 2025.

- Dataset: https://www.nuscenes.org/

Generative Models

27. Flow Matching for Image Generation

Difficulty: Hard

Description

Flow Matching is a recent generative modeling framework built on Continuous Normalizing Flows (CNFs). Unlike diffusion models that require simulating a stochastic process, Flow Matching trains a neural network to regress the vector field of a deterministic probability path that transforms noise into data. A particularly efficient variant uses Optimal Transport (OT) displacement interpolation, which produces straighter paths than the curved trajectories of diffusion, enabling faster sampling with fewer ODE solver steps.

Objectives

Implement a Flow Matching model from scratch using the conditional flow matching (CFM) objective. Train it on CIFAR-10 (for fast iteration) or CelebA 64×64. Report results in terms of FID, training wall-clock time, and number of ODE steps needed for good sample quality. Experiment with OT interpolation paths. Upon completing this project, we expect: (i) a brief description of the flow matching and diffusion literature; (ii) a working flow matching implementation; (iii) experimental comparisons of sample quality, training efficiency, and sampling speed.

References

- Lipman, Y., Chen, R.T.Q., Ben-Hamu, H., Nickel, M., Le, M. "Flow Matching for Generative Modeling." ICLR, 2023.
- Lipman, Y., Havasi, M., Holderrieth, P., et al. "Flow Matching Guide and Code." arXiv:2412.06264, 2024. (includes PyTorch library)
- Tong, A., Malkin, N., Huguet, G., et al. "Improving and Generalizing Flow-Based Generative Models with Minibatch Optimal Transport." TMLR, 2024.

Graphs & Structured Data

28. Molecule Property Prediction with Graph Neural Networks

Difficulty: Medium-Hard

Description

Molecules are naturally represented as graphs: atoms are nodes and chemical bonds are edges. Predicting molecular properties (solubility, toxicity, blood-brain barrier permeability) from this graph structure is a key task in drug discovery. Graph Neural Networks (GNNs) learn node-level representations through iterative message passing between neighbors, then aggregate them into a graph-level representation for property prediction.

Objectives

Use the MoleculeNet benchmark (BBBP, Tox21, ESOL), loadable via PyTorch Geometric or DeepChem. Implement at least two GNN variants (e.g., GCN and GIN) and compare their predictive performance. Use scaffold splitting (not random splitting). Evaluate using AUC-ROC for classification and RMSE for regression. Upon completing this project, we expect: (i) a brief description of the molecular ML and GNN literature; (ii) working GNN models with at least two architecture variants; (iii) experimental evaluations across multiple datasets.

References

- Gilmer, J., Schoenholz, S.S., Riley, P.F., Vinyals, O., Dahl, G.E. "Neural Message Passing for Quantum Chemistry." ICML, pp. 1263–1272, 2017.
- Wu, Z., Ramsundar, B., Feinberg, E.N., et al. "MoleculeNet: A Benchmark for Molecular Machine Learning." Chemical Science 9, pp. 513–530, 2018.
- Xu, K., Hu, W., Leskovec, J., Jegelka, S. "How Powerful are Graph Neural Networks?" ICLR, 2019. (GIN)
- PyTorch Geometric: https://pytorch-geometric.readthedocs.io/

OCR & Document Understanding

29. Historical Document Transcription

Difficulty: Medium-Hard

Description

Digitizing historical documents (manuscripts, letters, archives) requires OCR that can handle degraded ink, stains, faded text, unusual fonts, and non-standard layouts. Modern Transformer-based models like TrOCR reformulate the problem as an image-to-sequence task: a Vision Transformer encoder processes the text line image, and a language model decoder generates the text autoregressively.

Objectives

Use the IAM Handwriting Dataset (~1,500 pages) and/or the RIMES dataset (French handwriting). Implement or fine-tune a TrOCR-style model. Compare with a CTC-based baseline (CNN + LSTM + CTC loss). Evaluate using Character Error Rate (CER) and Word Error Rate (WER). Optionally test robustness by adding synthetic degradation. Upon completing this project, we expect: (i) a brief description of the OCR literature; (ii) a working transcription system; (iii) experimental evaluations comparing Transformer-based and CTC-based approaches.

References

- Li, M., Lv, T., Chen, J., et al. "TrOCR: Transformer-based Optical Character Recognition with Pre-trained Models." AAAI, 2023.
- Shi, B., Bai, X., Yao, C. "An End-to-End Trainable Neural Network for Image-Based Sequence Recognition." TPAMI 39(11), pp. 2298–2304, 2017. (CRNN+CTC)
- Dataset: https://fki.tic.heia-fr.ch/databases/iam-handwriting-database

---

Reinforcement Learning

30. Deep Reinforcement Learning for Game Playing

Difficulty: Very Hard

Description

Deep Reinforcement Learning (Deep RL) combines neural networks with trial-and-error learning: an agent interacts with an environment, receives rewards, and learns a policy that maximizes long-term cumulative reward. Deep Q-Networks (DQN) were the first to demonstrate human-level performance on Atari games by using a CNN to approximate the Q-value function directly from raw pixel inputs. Key innovations include experience replay and a target network.

Objectives

Use the Gymnasium Atari environments (e.g., Pong, Breakout, or Space Invaders). Implement a DQN agent from scratch: a CNN that takes stacked grayscale frames as input and outputs Q-values for each action. You must implement experience replay and a target network. Train on at least one game and plot the reward curve. Upon completing this project, we expect: (i) a brief description of the deep RL literature; (ii) a working DQN agent trained from scratch; (iii) reward curves, training analysis, and performance analysis.

References

- Mnih, V., Kavukcuoglu, K., Silver, D., et al. "Human-level Control through Deep Reinforcement Learning." Nature 518, pp. 529–533, 2015.
- Van Hasselt, H., Guez, A., Silver, D. "Deep Reinforcement Learning with Double Q-learning." AAAI, 2016. (Double DQN)
- Wang, Z., Schaul, T., Hessel, M., et al. "Dueling Network Architectures for Deep Reinforcement Learning." ICML, 2016. (Dueling DQN)
- Environment: https://gymnasium.farama.org/environments/atari/